

Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — one holistic "does this look okay?" check

```
# agents/approval_agent.py
from langgraph.types import interrupt
from langchain_openai import ChatOpenAI

def approval_node(state):
    draft = state["messages"][-1].content
    model = ChatOpenAI(model="gpt-4o-mini")
    verdict = model.invoke(f"Does this draft answer look okay to send? Draft: {dr

    if verdict.strip().lower().startswith("yes"):
        return {"approved": True}

    decision = interrupt({"draft": draft, "reason": "model flagged this as unclear"})
    return {"approved": decision.get("approved", False)}
```

Expected output, on almost any reasonably fluent draft:

```
approved=True    (the graph reaches END with no human ever consulted)
```

This technically wires `interrupt()` in and technically has a gate, but "does this look okay?" is exactly the vague, holistic judgment call the README warns about — a fluent-sounding draft with an unsupported claim in it will very often still get a "yes," for the same reason Project 2's first-draft Verifier over-approved: plausibility and correctness aren't the same thing, and a single vague question can't tell them apart.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — 2 concrete checks, and a real `interrupt()` /resume cycle

```
# agents/approval_agent.py
from langgraph.types import interrupt

RISKY_PHRASES = ["i've sent", "i have issued", "i've refunded", "i have deleted",

def check_draft_cites_context(draft: str, found_chunks: list) -> bool:
    combined_context = " ".join(c.page_content for c in found_chunks).lower()
    significant_words = [w for w in draft.lower().split() if len(w) > 5]
    return any(word in combined_context for word in significant_words)

def check_no_unsupported_action_language(draft: str) -> bool:
    return not any(phrase in draft.lower() for phrase in RISKY_PHRASES)

def approval_node(state):
    draft = state["messages"][-1].content
    cites_context = check_draft_cites_context(draft, state["found_chunks"])
    no_risky_language = check_no_unsupported_action_language(draft)

    if cites_context and no_risky_language:
        return {"approved": True}

    decision = interrupt({
        "draft": draft,
        "cites_context": cites_context,
        "no_risky_language": no_risky_language,
    })
    return {"approved": decision.get("approved", False)}
```

```
# graph.py (excerpt – Step 4's END replaced with the Approval node)
builder.add_node("approval", approval_node)
builder.add_edge("reason", "approval")
```

```
builder.add_edge("approval", END)
graph = builder.compile(checkpointer=MemorySaver())
```

```
# main.py
config = {"configurable": {"thread_id": "run-42"}}
result = graph.invoke({"task": "...", "messages": [...]}, config=config)
print(result)
# if this printed an __interrupt__ payload instead of a finished answer, the graph
# run resume.py next
```

```
# resume.py
from langgraph.types import Command
from graph import build_graph

graph = build_graph()
config = {"configurable": {"thread_id": "run-42"}}
result = graph.invoke(Command(resume={"approved": True}), config=config)
print(result["approved"], result["messages"][-1].content)
```

Expected output, `main.py` on a draft that fails `no_risky_language` (a test prompt engineered to make the Reasoner say "I've issued the refund"):

```
{'__interrupt__': [Interrupt(value={'draft': "I've issued the refund for your damaged item."})]}
```

Expected output, `resume.py` run right after, in a separate process, with the same `thread_id`:

```
True I've issued the refund for your damaged item.
```

Difference from Basic: the 2 checks are concrete and independently reasoned about, instead of one vague model question — `cites_context` catches ungrounded claims, `no_risky_language` catches a draft that implies an action actually happened. The resume cycle now genuinely uses `Command(resume=...)` against a real `thread_id`, proving `interrupt()` actually pauses and a later, separate call actually continues it — not just that the function compiles.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a named checklist instead of 2 inline variables

```
# agents/approval_agent.py
APPROVAL_CHECKS = [
    ("cites_context", lambda draft, chunks: check_draft_cites_context(draft, chunks)),
    ("no_risky_language", lambda draft, chunks: check_no_unsupported_action_language(draft, chunks))
]

def approval_node(state):
    draft = state["messages"][-1].content
    found_chunks = state["found_chunks"]

    results = {}
    for name, check in APPROVAL_CHECKS:
        results[name] = check(draft, found_chunks)

    if all(results.values()):
        return {"approved": True}

    failed = [name for name, ok in results.items() if not ok]
    decision = interrupt({"draft": draft, "failed_checks": failed})
    return {"approved": decision.get("approved", False)}
```

Expected output, on a draft that fails `no_risky_language` only:

```
failed_checks: ['no_risky_language']
```

Whoever reviews this interrupt (a human, or a log later) sees exactly which named check tripped, not just "something was unclear" — the same debuggability upgrade Project 2's Step 3 gave `MaxIterationsExceeded` .

Approach 2 — both branches proven with real tests

```
# test_project3.py
from unittest.mock import patch
```

```

from graph import build_graph
from langchain_core.documents import Document

def test_auto_approves_clean_draft():
    graph = build_graph()
    config = {"configurable": {"thread_id": "test-clean"}}
    result = graph.invoke({
        "task": "What is the return window for a damaged item?",
        "messages": [],
        "found_chunks": [Document(page_content="Damaged items can be returned with",
        "grounded": True,
    }, config=config)
    assert result.get("approved") is True
    assert "__interrupt__" not in result

def test_pauses_for_unsupported_draft():
    graph = build_graph()
    config = {"configurable": {"thread_id": "test-risky"}}
    # simulate a draft the Reasoner produced that uses risky action language,
    # bypassing the Reasoner call itself to isolate the Approval node's own logic
    with patch("agents.reasoner_agent.reason_node", return_value={
        "messages": [AIMessage(content="I've issued the refund already.")]
    }):
        result = graph.invoke({
            "task": "Can I get a refund for my damaged item?",
            "messages": [],
            "found_chunks": [Document(page_content="Damaged items can be returned",
            "grounded": True,
        }, config=config)
    assert "__interrupt__" in result

```

Expected output:

```

test_auto_approves_clean_draft PASSED
test_pauses_for_unsupported_draft PASSED

```

Both passing together is what proves the gate genuinely branches both ways — a version of `approval_node` that always returns `{"approved": True}` would still pass the first test alone.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's 2 checks work, but they're 2 local variables with no shared structure — adding a third check later means editing the function body in 2 more places (the call, and the `if`). Approach 1 makes the checklist a real list that's easy to extend and easy to report on by name. Approach 2 is the concrete

proof, matching this project's pattern from every prior step, that both the "approves on its own" and "pauses for a human" paths are real and tested, not just theoretically possible.

Which one should you actually write? All of Approach 1's named-checklist structure — it's what keeps this gate maintainable as more checks get added over time, and it's what turns "the human reviewer sees a vague flag" into "the human reviewer sees exactly which rule tripped and why." Approach 2's 2 tests are the actual final item on the README's own checklist ("both paths tested") — treat them as required, not optional, the same way Project 2's Verifier needed proof it could actually reject something.